

Databases Fundamentals

Die Grundlage für persistente Daten und stabile Geschäftsdomänen

01

| SQL vs. NoSQL

Die zwei grundlegenden Paradigmen der Datenspeicherung

Vergleich SQL und NoSQL

Datenbanken werden grundlegend in **relationale (SQL)** und **nicht-tabellarische (NoSQL)** Systeme unterteilt. Im echten Entwickleralltag überwiegt meist der relationale Ansatz, da sich Business-Logiken hervorragend über feste Beziehungen abbilden lassen.

SQL (Relational)

- Basiert auf strikten **Beziehungen**
- Festes, vordefiniertes **Schema**
- Starke Datenintegrität
- Schwieriger horizontal zu skalieren
- **Tech-Stack:** PostgreSQL, MySQL

NoSQL (Non-Tabular)

- Keine starre Form (**Schemalos**)
- Eingeschränkte Beziehungslogik
- Sehr hohe Performance
- Hervorragend horizontal skalierbar
- **Tech-Stack:** MongoDB, Neo4j

Wann nutzt man was?

Die Wahl des Datenbanksystems hängt von den Anforderungen der Anwendung ab:

Kriterium	SQL-Datenbanken	NoSQL-Datenbanken
Fokus	Komplexe Beziehungen wichtig	Maximale Performance & Durchsatz
Schema	Benötigt ein verlässliches Schema	Flexibel für unstrukturierte Daten
Beispiele	E-Commerce-Systeme, Banking	CMS, Echtzeitdaten, Log-Storage

02

| SQL Grundlagen

Struktur, Abfragelogik und die Anatomie einer Query

SQL – Was ist das?

Im Kern ist SQL eine strukturierte Tabelle mit festen Spalten.

Unsere Beispieltabelle: `users`. Alle folgenden Beispiele arbeiten mit derselben Tabelle.

id	first_name	age	status
1	Anna	22	active
2	Ben	16	active
3	Clara	30	inactive

Jede Zeile repräsentiert einen Datensatz.

Jede Spalte beschreibt ein bestimmtes Attribut.

CRUD – Die vier Grundoperationen

Jede Anwendung arbeitet permanent mit diesen vier Aktionen:

Operation	Bedeutung
CREATE	Daten erstellen
READ	Daten lesen
UPDATE	Daten aktualisieren
DELETE	Daten löschen

● CREATE – Daten erstellen

JavaScript

```
users.push({  
  id: 4,  
  first_name: "Tom",  
  age: 20,  
  status: "active",  
});
```

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
+ { id: 4, first_name: "Tom", age: 20, status: "active" },  
];
```

SQL

```
INSERT INTO users (id, first_name, age, status)
VALUES (4, 'Tom', 20, 'active');
```

	id	first_name	age	status
	---	-----	---	-----
	1	Anna	22	active
	2	Ben	16	active
	3	Clara	30	inactive
+	4	Tom	20	active

➡ SQL macht exakt dieselbe Operation persistent in der Datenbank.

● READ – Daten lesen

JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
// Hole Namen aller User  
const result = users.map((u) => ({ first_name: u.first_name }));  
// result = [{ first_name: "Anna" }, { first_name: "Ben" }, { first_name: "Clara" }]
```

➔ `map()` = bestimmte Felder zurückgeben

SQL

Die Anatomie einer Standard-SQL-Abfrage kann aus diesen Klauseln bestehen:

- **SELECT**: Bestimmt, **welche Spalten** zurückgegeben werden.
- **FROM**: Bestimmt, aus **welcher Tabelle** gelesen wird.

```
-- Hole Namen aller User  
SELECT first_name FROM users
```

first_name
Anna
Ben
Clara

oder alle Spalten

```
SELECT * FROM users
```

Weitere Elemente einer Read Operation

Die Anatomie einer SQL-Abfrage:

Klausel	Bedeutung
SELECT	Welche Spalten sollen zurückgegeben werden?
FROM	Aus welcher Tabelle wird gelesen?
WHERE	Welche Zeilen sollen gefiltert werden?
ORDER BY	Wie sollen Ergebnisse sortiert werden?
LIMIT	Wie viele Ergebnisse sollen zurückkommen?

JavaScript

```
const users = [  
  { id: 1, first_name: "Anna", age: 22, status: "active" },  
  { id: 2, first_name: "Ben", age: 16, status: "active" },  
  { id: 3, first_name: "Clara", age: 30, status: "inactive" },  
];  
  
const result = users  
  .filter((u) => u.age > 18)  
  .sort((a, b) => b.age - a.age)  
  .slice(0, 5)  
  .map((u) => ({ first_name: u.first_name }));  
  
// result = [{ first_name: "Clara" }, { first_name: "Anna" }]
```

➡ `filter()` = Datensätze auswählen

SQL

id	first_name	age	status
---	-----	---	-----
1	Anna	22	active
2	Ben	16	active
3	Clara	30	inactive

```
SELECT first_name FROM users WHERE age > 18 ORDER BY age DESC LIMIT 2
```

first_name

Clara
Anna

● UPDATE – Daten aktualisieren

```
UPDATE users  
SET age = 17  
WHERE id = 2;
```

id	first_name	age
1	Anna	22
- 2	Ben	16
+ 2	Ben	17
3	Clara	30

➔ Wichtig: Ohne `WHERE` würden alle Datensätze geändert werden.

● DELETE – Daten löschen

```
DELETE FROM users  
WHERE id = 2;
```

id	first_name
---	-----
1	Anna
- 2	Ben
3	Clara

➡ Der Datensatz wird vollständig entfernt.

⚠ DELETE ohne WHERE löscht alle Datensätze der Tabelle.

Zusammenfassung

SQL ist keine neue Denkweise

Das kennt ihr bereits aus JavaScript

SQL	JavaScript
INSERT	<code>push()</code>
SELECT	<code>map()</code>
WHERE	<code>filter()</code>
UPDATE	<code>map() + condition</code>
DELETE	<code>filter()</code>

Beispiel

```
SELECT first_name  
FROM users  
WHERE age > 18;
```

JavaScript Denkweise dazu

```
users  
  .filter((u) => u.age > 18)  
  .map((u) => ({  
    first_name: u.first_name,  
  }));
```

JavaScript Arrays

Problem	Konsequenz
Array existiert nur im RAM des laufenden Prozesses	→ Kein Mechanismus schreibt den Zustand automatisch dauerhaft weg
Prozess terminiert (Neustart, Crash, Deployment)	→ Kompletter Datenverlust, da nichts persistiert wurde
Suche im Array ist linear ($O(n)$)	→ Bei Millionen Datensätzen wird jede Abfrage spürbar langsamer, bis das System unbenutzbar wird
Array lebt nur in einer einzelnen Prozessinstanz	→ Bei mehreren Server-Instanzen (Skalierung) hat jede Instanz ihren eigenen, widersprüchlichen Datenstand

Warum Datenbanken das lösen

Problem (ohne DB)	Konsequenz (mit DB gelöst)
Keine dauerhafte Speicherung	Persistenz: Daten liegen auf Disk und überleben Neustarts und Abstürze
Lineare Suche skaliert nicht	Performance: Indexstrukturen machen Zugriffe auch bei riesigen Datenmengen schnell
Kein Zugriffsschutz, keine Nachvollziehbarkeit	Sicherheit: Rollenbasierte Zugriffskontrolle, Verschlüsselung, Audit-Logs
Mehrere Prozesse haben je eigenen, inkonsistenten Zustand	Zentrale Datenquelle: Alle Instanzen greifen auf denselben, konsistenten Datenbestand zu



Aufgabe: Basic Read Operationen üben

- Rufen Sie <https://www.sql-practice.com/> auf
- Üben Sie SQL Kommandos

03

Beziehungen zwischen Tabellen

Relationale Datenbanken werden mächtig, sobald Tabellen miteinander verbunden werden.

Warum brauchen wir mehrere Tabellen?

Statt Daten mehrfach zu speichern, speichern relationale Datenbanken Beziehungen.

❌ Schlechte Lösung

Order	User
Laptop	Anna
Mouse	Anna

Der Name "Anna" wird mehrfach gespeichert.

✓ Gute Lösung

users

id	name
1	Anna

orders

id	user_id	product
1	1	Laptop
2	1	Mouse

➡ Daten werden normalisiert gespeichert.

Primary Key & Foreign Key

Primary Key (PK)

- Eindeutiger Bezeichner pro Zeile
- Darf nie `NULL` oder doppelt sein
- Meist eine Auto-Increment-ID
- Beispiel: `users.id`

Foreign Key (FK)

- Verweis auf einen PK in einer anderen Tabelle
- Stellt referenzielle Integrität sicher
- Definiert die Beziehung zwischen Entitäten
- Beispiel: `orders.user_id → users.id`

Beispiel: Users & Orders

users

id	name
1	Anna
2	Ben

orders

id	user_id	product
1	1	Laptop
2	1	Mouse
3	2	Keyboard

`orders.user_id` verweist auf `users.id` → das ist ein Foreign Key (FK)

Beziehungstypen

Typ	Beispiel
1:1	User ↔ Profile
1:n	User ↔ Orders
n:m	Students ↔ Courses

One-to-Many (1:n)



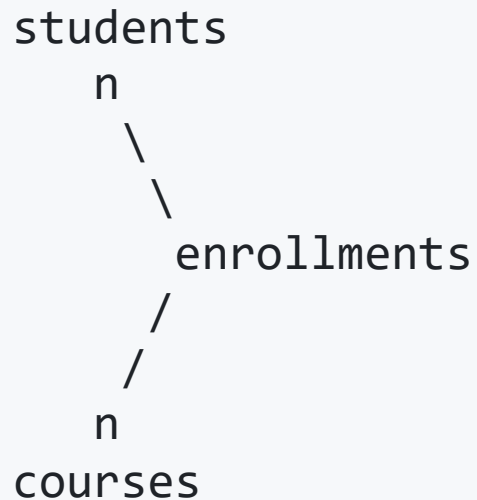
- Ein User kann **viele** Orders besitzen → 1:n (One-to-Many)
- Jede Order gehört **genau einem** User
- **FK** = Foreign Key (Verweis auf andere Tabelle)

Many-to-Many (n:m)

Ein Student kann viele Kurse besuchen.

Ein Kurs hat viele Studenten.

Direkte Speicherung ist nicht möglich.



➡ Dafür benötigt man eine Zwischentabelle.

Many-to-Many Beispiel

students

id	name
1	Anna
2	Ben

courses

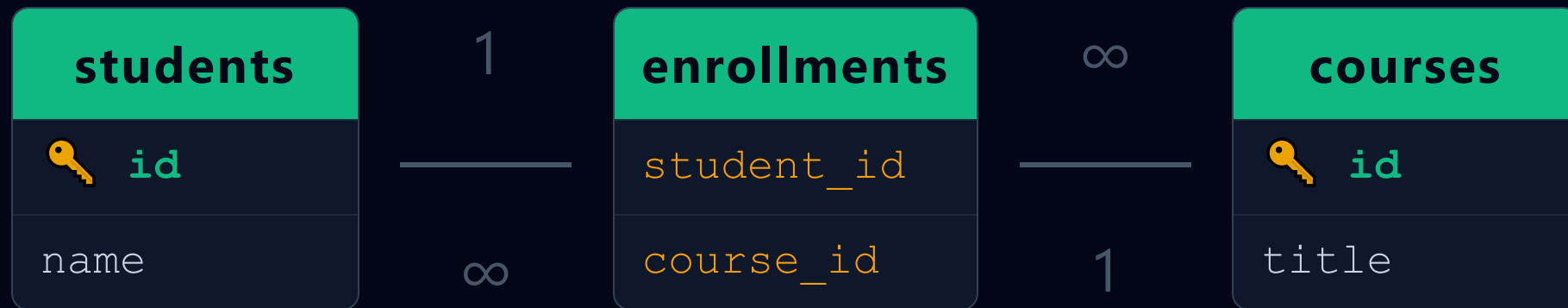
id	title
1	SQL
2	React

enrollments

student_id	course_id
1	1
1	2
2	1

Anna besucht den SQL-Kurs und den React-Kurs. Ben besucht nur den SQL-Kurs.

ER-Diagramm für Many-to-Many



➔ Eine n:m Beziehung wird immer über eine Zwischentabelle modelliert.

One-to-One (1:1)

Ein Datensatz aus Tabelle A ist mit genau **einem** Datensatz aus Tabelle B verknüpft.

Häufige Anwendungsfälle:

- **Performance:** Auslagerung von selten genutzten, großen Texten.
- **Sicherheit:** Isolierte Speicherung sensibler Profildaten (z.B. User-Credentials getrennt von öffentlichen Infos).
- **Struktur:** Logische Aufteilung einer zu großen Entität.

One-to-One Beispiel

users

id	name
1	Anna
2	Ben

profiles

id	user_id	website
10	1	anna.dev
11	2	ben.codes

➡ Der Foreign Key `profiles.user_id` verweist auf `users.id`.

Damit es eine 1:1 Beziehung bleibt, muss diese Spalte **UNIQUE** sein!

ER-Diagramm für One-to-One

- Ein User hat **genau ein** Profil.
- Jedes Profil gehört zu **genau einem** User.
- Das `UNIQUE`-Constraint auf dem Foreign Key verhindert Duplikate.



Aufgabe: ER-Diagramme

JOIN – Tabellen verbinden

Mit `JOIN` kombiniert SQL Daten aus mehreren Tabellen.

```
SELECT users.name, orders.product
FROM users
JOIN orders
ON users.id = orders.user_id;
```

Ergebnis

name	product
Anna	Laptop
Anna	Mouse
Ben	Keyboard

JavaScript Denkweise dazu

```
const result = users.flatMap((u) =>
  orders
    .filter((o) => o.user_id === u.id)
    .map((o) => ({ name: u.name, product: o.product })),
);
```

➡ `JOIN` entspricht hier einer Kombination aus `filter()` und `map()` über beide Arrays.

INNER JOIN vs. LEFT JOIN

INNER JOIN – nur Schnittmenge

```
SELECT users.name, orders.product
FROM users
INNER JOIN orders
ON users.id = orders.user_id;
```

name	product
Anna	Laptop
Ben	Keyboard

✗ Clara – keine Orders → fällt raus

LEFT JOIN – alle linken Zeilen

```
SELECT users.name, orders.product
FROM users
LEFT JOIN orders
ON users.id = orders.user_id;
```

name	product
Anna	Laptop
Ben	Keyboard
Clara	NULL

✓ Clara – keine Orders → NULL

JOIN über mehrere Tabellen

Welche Kurse besucht Anna?

```
SELECT students.name, courses.title
FROM students
JOIN enrollments
    ON students.id = enrollments.student_id
JOIN courses
    ON courses.id = enrollments.course_id;
```

Ergebnis

name	title
Anna	SQL
Anna	React
Ben	SQL

Aufgabe: JOIN üben

- gehen Sie auf <https://www.sql-practice.com/>
- filtern Sie nach JOIN

04

| Zusammenfassung

Wichtig für echte Anwendungen

Datenbanken sind nicht nur Speicher.

Sie modellieren:

- Geschäftslogik
- Beziehungen
- Zustände
- Historie
- Sicherheit
- Konsistenz

Zusammenfassung

- SQL Datenbank = strukturierte relationale Daten
- NoSQL Datenbank = flexible hochskalierbare Daten
- CRUD = Basis jeder Anwendung
- JOINS verbinden Geschäftslogik
- Datenabfragen sind reine Logik
- Gute Datenmodelle sind wichtiger als Frameworks